

```
In [39]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from scipy import stats
from sklearn.preprocessing import StandardScaler

import seaborn as sns

from sklearn.decomposition import PCA

from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

```
In [12]: import warnings
warnings.filterwarnings('ignore')
```

```
In [13]: songs_cohort_df = pd.read_csv('data/rolling_stones_spotify.csv')
songs_cohort_df.head()
```

```
Out[13]:
```

	Unnamed: 0	name	album	release_date	track_number	
0	0	Concert Intro Music - Live	Licked Live In NYC	2022-06-10	1	2IEkywLJ4ykbhi1yRQv
1	1	Street Fighting Man - Live	Licked Live In NYC	2022-06-10	2	6GVgVJBKkGJoRfarYRv
2	2	Start Me Up - Live	Licked Live In NYC	2022-06-10	3	1Lu761pZ0dBTGpzxaQo
3	3	If You Can't Rock Me - Live	Licked Live In NYC	2022-06-10	4	1agTQzOTUnGNggycEc
4	4	Don't Stop - Live	Licked Live In NYC	2022-06-10	5	7piGJR8YndQBQWVXv6k

```
In [14]: songs_cohort_df = songs_cohort_df.drop(['Unnamed: 0'],axis=1)
songs_cohort_df
```

Out [14]:

	name	album	release_date	track_number	id	
0	Concert Intro Music - Live	Licked Live In NYC	2022-06-10	1	2IEkywLJ4ykbhi1yRQvmsT	
1	Street Fighting Man - Live	Licked Live In NYC	2022-06-10	2	6GVgVJBKkGJoRfarYRvGTU	s
2	Start Me Up - Live	Licked Live In NYC	2022-06-10	3	1Lu761pZ0dBTGpzxaQoZNW	st
3	If You Can't Rock Me - Live	Licked Live In NYC	2022-06-10	4	1agTQzOTUnGNggycEqiDH	s
4	Don't Stop - Live	Licked Live In NYC	2022-06-10	5	7piGJR8YndQBQWVXv6KtQw	sp
...	...	...	...	...	...	...
1605	Carol	The Rolling Stones	1964-04-16	8	08l7M5UpRnffGI0FyuRiQZ	
1606	Tell Me	The Rolling Stones	1964-04-16	9	3JZIIQBstM6WwoJdzFDLhx	s
1607	Can I Get A Witness	The Rolling Stones	1964-04-16	10	0t2qvfSBQ3Y08lzRRoVTdb	
1608	You Can Make It If You Try	The Rolling Stones	1964-04-16	11	5ivls5vwSj0RChOlviY3On	
1609	Walking The Dog	The Rolling Stones	1964-04-16	12	43SkTJJ2xleDaeiE4TIM70	

1610 rows x 17 columns

In [15]: `songs_cohort_df.shape`

Out [15]: `(1610, 17)`

In [16]: `songs_cohort_df.info()`

```

<class 'pandas.DataFrame'>
RangeIndex: 1610 entries, 0 to 1609
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  -
0   name                   1610 non-null   str
1   album                  1610 non-null   str
2   release_date           1610 non-null   str
3   track_number           1610 non-null   int64
4   id                     1610 non-null   str
5   uri                    1610 non-null   str
6   acousticness            1610 non-null   float64
7   danceability            1610 non-null   float64
8   energy                  1610 non-null   float64
9   instrumentalness        1610 non-null   float64
10  liveness                1610 non-null   float64
11  loudness                1610 non-null   float64
12  speechiness             1610 non-null   float64
13  tempo                   1610 non-null   float64
14  valence                 1610 non-null   float64
15  popularity              1610 non-null   int64
16  duration_ms             1610 non-null   int64
dtypes: float64(9), int64(3), str(5)
memory usage: 405.1 KB

```

```
In [17]: songs_cohort_df.describe()
```

```
Out[17]:
```

	track_number	acousticness	danceability	energy	instrumentalness
count	1610.000000	1610.000000	1610.000000	1610.000000	1610.000000
mean	8.613665	0.250475	0.468860	0.792352	0.164170
std	6.560220	0.227397	0.141775	0.179886	0.276249
min	1.000000	0.000009	0.104000	0.141000	0.000000
25%	4.000000	0.058350	0.362250	0.674000	0.000219
50%	7.000000	0.183000	0.458000	0.848500	0.013750
75%	11.000000	0.403750	0.578000	0.945000	0.179000
max	47.000000	0.994000	0.887000	0.999000	0.996000

1. Initial data inspection and data cleaning:

a. Examine the data initially to identify duplicates, missing values, irrelevant entries, or outliers. Check for any instances of erroneous entries and rectify them as needed

```
In [18]: songs_cohort_df.isnull().sum()
```

```
Out[18]: name          0
         album         0
         release_date  0
         track_number  0
         id            0
         uri           0
         acousticness  0
         danceability  0
         energy        0
         instrumentalness 0
         liveness     0
         loudness      0
         speechiness  0
         tempo         0
         valence       0
         popularity   0
         duration_ms   0
         dtype: int64
```

```
In [19]: songs_cohort_df.duplicated().sum()
```

```
Out[19]: np.int64(0)
```

```
In [20]: songs_cohort_df = songs_cohort_df.drop_duplicates()
         songs_cohort_df
```

Out [20]:

	name	album	release_date	track_number		id
0	Concert Intro Music - Live	Licked Live In NYC	2022-06-10	1	2IEkywLJ4ykbhi1yRQvmsT	
1	Street Fighting Man - Live	Licked Live In NYC	2022-06-10	2	6GVgVJBKkGJoRfarYRvGTU	s
2	Start Me Up - Live	Licked Live In NYC	2022-06-10	3	1Lu761pZ0dBTGpzxaQoZNW	st
3	If You Can't Rock Me - Live	Licked Live In NYC	2022-06-10	4	1agTQzOTUnGNggycEqiDH	s
4	Don't Stop - Live	Licked Live In NYC	2022-06-10	5	7piGJR8YndQBQWVXv6KtQw	sp
...	...	...	...	...	...	...
1605	Carol	The Rolling Stones	1964-04-16	8	08l7M5UpRnffGI0FyuRiQZ	
1606	Tell Me	The Rolling Stones	1964-04-16	9	3JZIIQBstM6WwoJdzFDLhx	s
1607	Can I Get A Witness	The Rolling Stones	1964-04-16	10	0t2qvfSBQ3Y08lzRRoVTdb	
1608	You Can Make It If You Try	The Rolling Stones	1964-04-16	11	5ivls5vwSj0RChOlviY3On	
1609	Walking The Dog	The Rolling Stones	1964-04-16	12	43SkTJJ2xleDaeiE4TIM70	

1610 rows × 7 columns

```
In [21]: songs_cohort_df['release_date'] = pd.to_datetime(songs_cohort_df['release_date'])
songs_cohort_df['release_year'] = songs_cohort_df['release_date'].dt.year
songs_cohort_df
```

Out [21]:

	name	album	release_date	track_number		id
0	Concert Intro Music - Live	Licked Live In NYC	2022-06-10	1	2IEkywLJ4ykbhi1yRQvmsT	
1	Street Fighting Man - Live	Licked Live In NYC	2022-06-10	2	6GVgVJBKkGJoRfarYRvGTU	s
2	Start Me Up - Live	Licked Live In NYC	2022-06-10	3	1Lu761pZ0dBTGpzxaQoZNW	st
3	If You Can't Rock Me - Live	Licked Live In NYC	2022-06-10	4	1agTQzOTUnGNgyckEqiDH	s
4	Don't Stop - Live	Licked Live In NYC	2022-06-10	5	7piGJR8YndQBQWVXv6KtQw	sp
...	...	...	...	...	...	...
1605	Carol	The Rolling Stones	1964-04-16	8	08l7M5UpRnffGI0FyuRiQZ	
1606	Tell Me	The Rolling Stones	1964-04-16	9	3JZIIQBstM6WwoJdzFDLhx	s
1607	Can I Get A Witness	The Rolling Stones	1964-04-16	10	0t2qvfSBQ3Y08lzRRoVTdb	
1608	You Can Make It If You Try	The Rolling Stones	1964-04-16	11	5ivls5vwSj0RChOlviY3On	
1609	Walking The Dog	The Rolling Stones	1964-04-16	12	43SkTJJ2xleDaeiE4TIM70	

1610 rows × 18 columns

```
In [22]: numeric_cols = ['acousticness', 'danceability', 'energy', 'instrumentalness']
scaler = StandardScaler()
songs_cohort_df[numeric_cols] = scaler.fit_transform(songs_cohort_df[numeric_cols])
```

Out [22]:

	name	album	release_date	track_number		id
0	Concert Intro Music - Live	Licked Live In NYC	2022-06-10	1	2IEkywLJ4ykbhi1yRQvmsT	
1	Street Fighting Man - Live	Licked Live In NYC	2022-06-10	2	6GVgVJBKkGJoRfarYRvGTU	s
2	Start Me Up - Live	Licked Live In NYC	2022-06-10	3	1Lu761pZ0dBTGpzxaQoZNW	st
3	If You Can't Rock Me - Live	Licked Live In NYC	2022-06-10	4	1agTQzOTUnGNggycEqiDH	s
4	Don't Stop - Live	Licked Live In NYC	2022-06-10	5	7piGJR8YndQBQWVXv6KtQw	sp
...	...	...	...	...	...	...
1605	Carol	The Rolling Stones	1964-04-16	8	08l7M5UpRnffGI0FyuRiQZ	
1606	Tell Me	The Rolling Stones	1964-04-16	9	3JZIIQBstM6WwoJdzFDLhx	s
1607	Can I Get A Witness	The Rolling Stones	1964-04-16	10	0t2qvfsBQ3Y08lzRRoVTdb	
1608	You Can Make It If You Try	The Rolling Stones	1964-04-16	11	5ivls5vwSj0RChOlviY3On	
1609	Walking The Dog	The Rolling Stones	1964-04-16	12	43SkTJJ2xleDaeiE4TIM70	

1610 rows × 18 columns

In [23]:

```
songs_cohort_df = songs_cohort_df[(np.abs(stats.zscore(songs_cohort_df[  
songs_cohort_df
```

Out [23]:

	name	album	release_date	track_number		id	
1	Street Fighting Man - Live	Licked Live In NYC	2022-06-10	2	6GVgVJBKkGJoRfarYRvGTU	s	
2	Start Me Up - Live	Licked Live In NYC	2022-06-10	3	1Lu761pZ0dBTGpzxaQoZNW	sp	
3	If You Can't Rock Me - Live	Licked Live In NYC	2022-06-10	4	1agTQzOTUnGNggycEqiDH	s	
4	Don't Stop - Live	Licked Live In NYC	2022-06-10	5	7piGJR8YndQBQWVXv6KtQw	sp	
5	Monkey Man - Live	Licked Live In NYC	2022-06-10	6	3ixZ1NQk5sqvBZgWeldiqr		
...	...	...	...	...	...	...	
1605	Carol	The Rolling Stones	1964-04-16	8	08l7M5UpRnffGI0FyuRiQZ		
1606	Tell Me	The Rolling Stones	1964-04-16	9	3JZIIQBstM6WwoJdzFDLhx	s	
1607	Can I Get A Witness	The Rolling Stones	1964-04-16	10	0t2qvfsBQ3Y08lzRRoVTdb		
1608	You Can Make It If You Try	The Rolling Stones	1964-04-16	11	5ivls5vwSj0RChOlviY3On		
1609	Walking The Dog	The Rolling Stones	1964-04-16	12	43SkTJJ2xleDaeiE4TIM70		

1508 rows × 18 columns

In [24]: `songs_cohort_df.shape`

Out [24]: `(1508, 18)`

In [25]: `songs_cohort_df['album'] = songs_cohort_df['album'].str.title()
songs_cohort_df`

Out [25]:

	name	album	release_date	track_number		id	
1	Street Fighting Man - Live	Licked Live In Nyc	2022-06-10	2	6GVgVJBKkGJoRfarYRvGTU	s	
2	Start Me Up - Live	Licked Live In Nyc	2022-06-10	3	1Lu761pZ0dBTGpzxaQoZNW	sp	
3	If You Can't Rock Me - Live	Licked Live In Nyc	2022-06-10	4	1agTQzOTUnGNggycEqiDH	s	
4	Don't Stop - Live	Licked Live In Nyc	2022-06-10	5	7piGJR8YndQBQWVXv6KtQw	sp	
5	Monkey Man - Live	Licked Live In Nyc	2022-06-10	6	3ixZ1NQk5sqvBZgWeldiqr		
...	...	...	...	...	...	...	
1605	Carol	The Rolling Stones	1964-04-16	8	08l7M5UpRnffGI0FyuRiQZ		
1606	Tell Me	The Rolling Stones	1964-04-16	9	3JZIIQBstM6WwoJdzFDLhx	s	
1607	Can I Get A Witness	The Rolling Stones	1964-04-16	10	0t2qvfsBQ3Y08lzRRoVTdb		
1608	You Can Make It If You Try	The Rolling Stones	1964-04-16	11	5ivls5vwSj0RChOlviY3On		
1609	Walking The Dog	The Rolling Stones	1964-04-16	12	43SkTJJ2xleDaeiE4TIM70		

1508 rows × 18 columns

In [26]: `songs_cohort_df.info()`

```

<class 'pandas.DataFrame'>
Index: 1508 entries, 1 to 1609
Data columns (total 18 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   name                  1508 non-null   str    
 1   album                 1508 non-null   str    
 2   release_date          1508 non-null   datetime64[us]
 3   track_number          1508 non-null   int64  
 4   id                    1508 non-null   str    
 5   uri                   1508 non-null   str    
 6   acousticness          1508 non-null   float64
 7   danceability          1508 non-null   float64
 8   energy                1508 non-null   float64
 9   instrumentalness       1508 non-null   float64
10   liveness              1508 non-null   float64
11   loudness              1508 non-null   float64
12   speechiness           1508 non-null   float64
13   tempo                 1508 non-null   float64
14   valence               1508 non-null   float64
15   popularity            1508 non-null   float64
16   duration_ms           1508 non-null   float64
17   release_year          1508 non-null   int32  
dtypes: datetime64[us](1), float64(11), int32(1), int64(1), str(4)
memory usage: 383.1 KB

```

3. Perform exploratory data analysis and feature engineering

a. Utilize suitable visualizations to identify the two albums that should be recommended to anyone based on the number of popular songs in each album

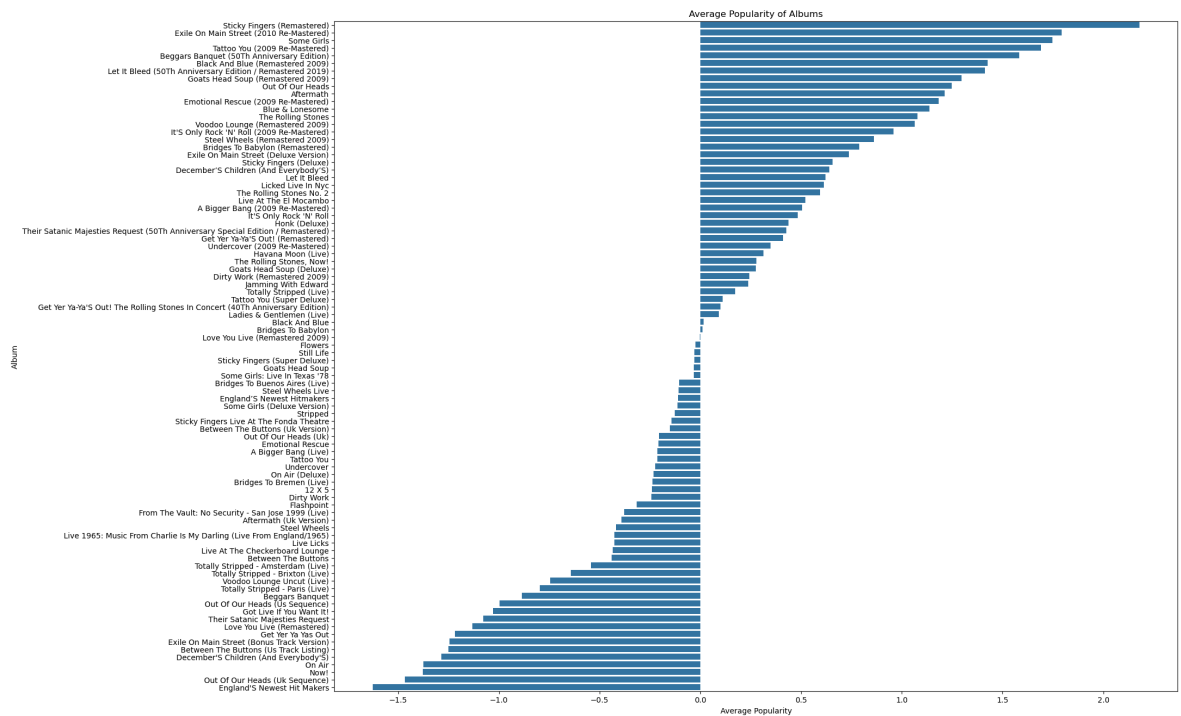
```

In [27]: album_popularity = songs_cohort_df.groupby('album')['popularity'].mean().

plt.figure(figsize=(20,16))
sns.barplot(x=album_popularity.values, y=album_popularity.index)
plt.title('Average Popularity of Albums')
plt.xlabel('Average Popularity')
plt.ylabel('Album')
plt.show()

# Identify the two albums with the most popular songs
top_two_albums = album_popularity.head(2)
print(f"The top two albums recommended based on the number of popular son

```



The top two albums recommended based on the number of popular songs are :

album	popularity
Sticky Fingers (Remastered)	2.178938
Exile On Main Street (2010 Re-Mastered)	1.792693

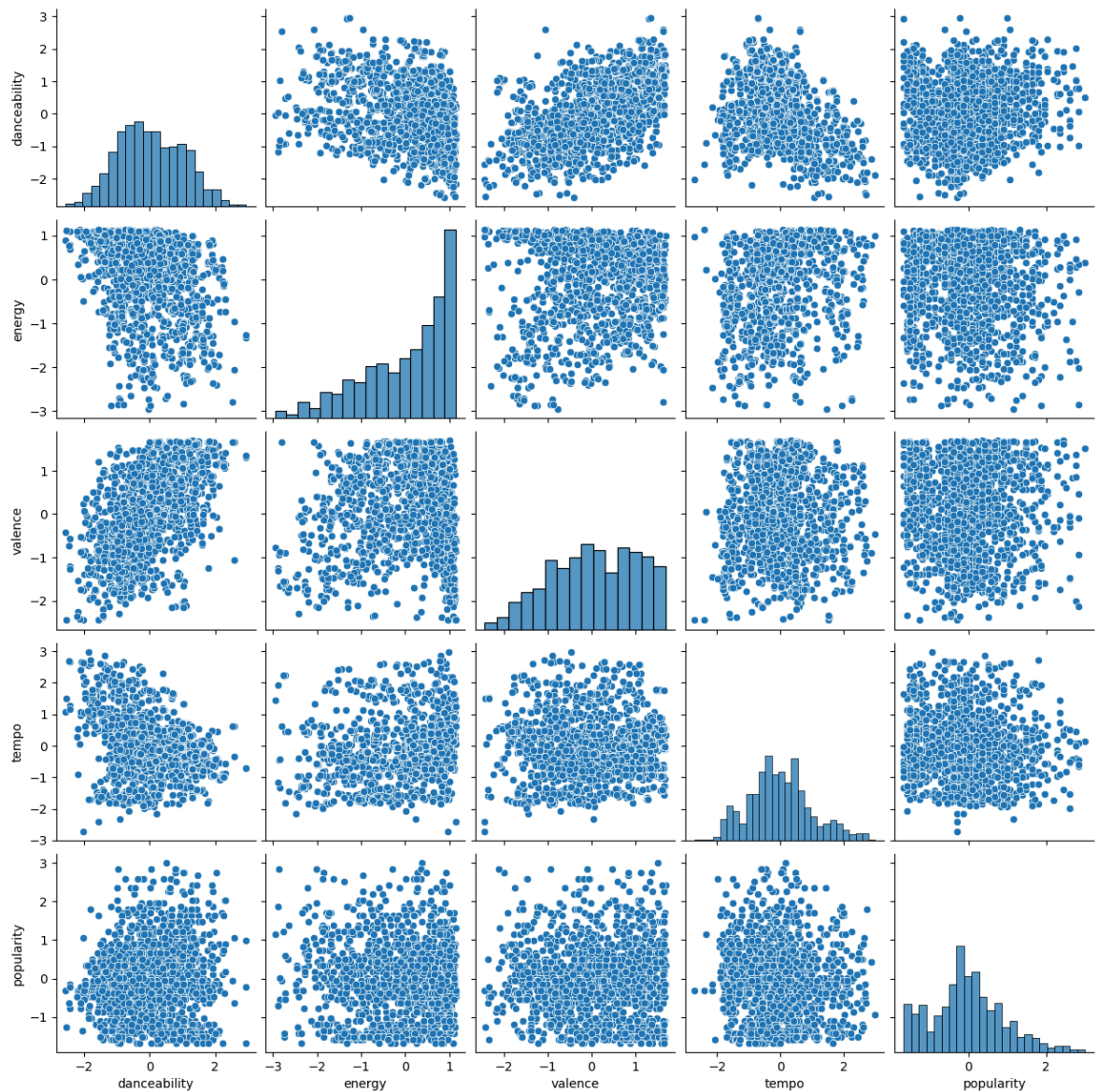
Name: popularity, dtype: float64

b. Conduct exploratory data analysis to delve into various features of songs, aiming to identify patterns

```
In [28]: plt.figure(figsize=(12,10))
sns.pairplot(songs_cohort_df[['danceability','energy', 'valence', 'tempo'])
```

Out[28]: <seaborn.axisgrid.PairGrid at 0x3636b3770>

<Figure size 1200x1000 with 0 Axes>



```
In [29]: numeric_cols = [
    'acousticness', 'danceability', 'energy', 'instrumentalness',
    'liveness', 'loudness', 'speechiness', 'tempo', 'valence',
    'popularity', 'duration_ms'
]

df_numeric = songs_cohort_df[numeric_cols].copy()

for col in df_numeric.columns:
    df_numeric[col] = pd.to_numeric(df_numeric[col], errors='coerce')

df_numeric = df_numeric.dropna()

corr_matrix = df_numeric.corr()

print("\nCorrelation Matrix:")
corr_matrix
```

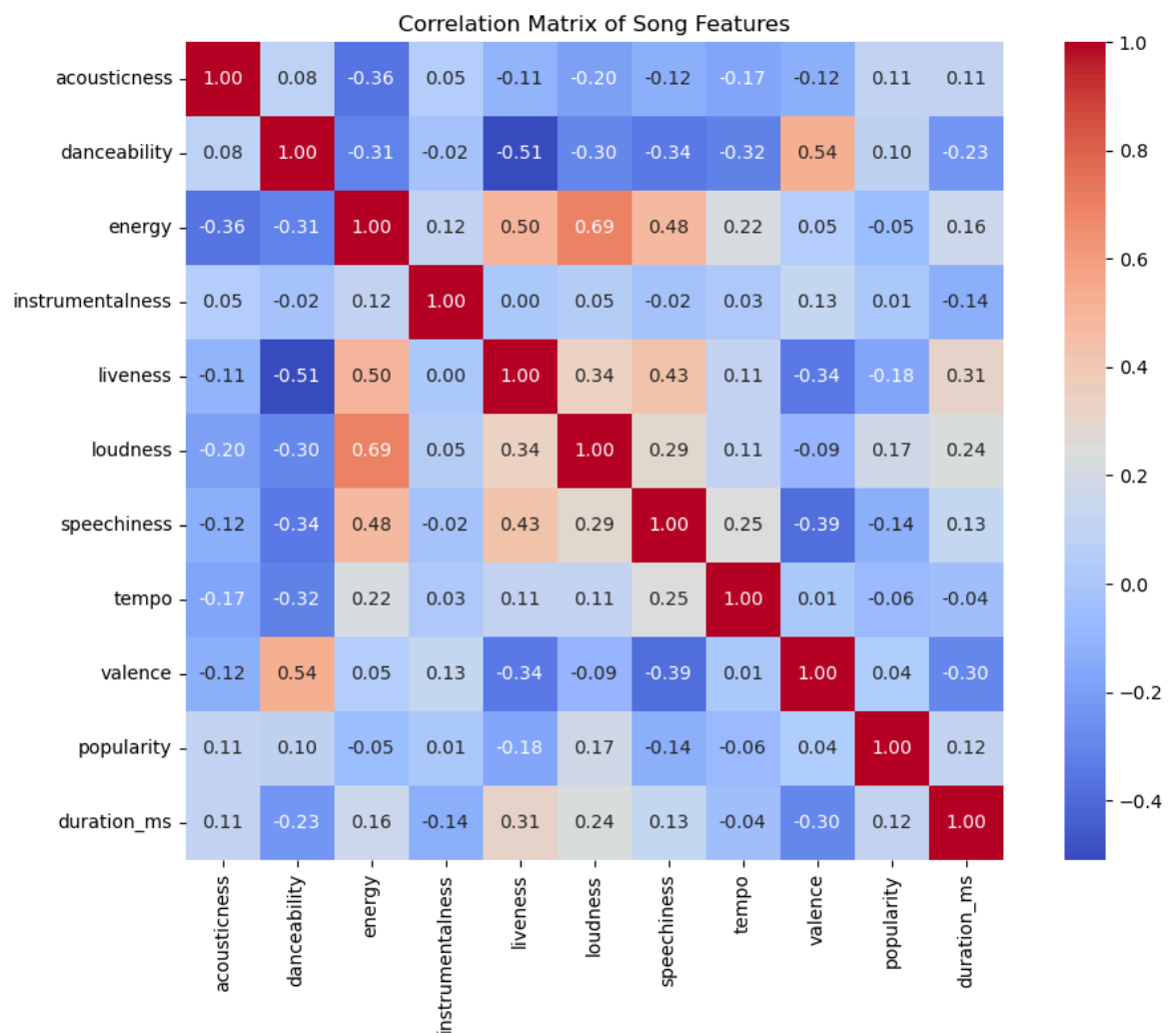
Correlation Matrix:

Out [29]:

	acousticness	danceability	energy	instrumentalness	liveness
acousticness	1.000000	0.081612	-0.363625	0.046211	-0.112355
danceability	0.081612	1.000000	-0.312081	-0.024561	-0.510922
energy	-0.363625	-0.312081	1.000000	0.122720	0.503274
instrumentalness	0.046211	-0.024561	0.122720	1.000000	0.001311
liveness	-0.112355	-0.510922	0.503274	0.001311	1.000000
loudness	-0.200142	-0.297296	0.694790	0.050053	0.340865
speechiness	-0.119025	-0.342865	0.479397	-0.015671	0.429685
tempo	-0.173219	-0.318273	0.215697	0.029341	0.113491
valence	-0.115866	0.535234	0.046665	0.127868	-0.341861
popularity	0.105469	0.099812	-0.051860	0.011834	-0.180035
duration_ms	0.106276	-0.233560	0.156284	-0.138705	0.313681

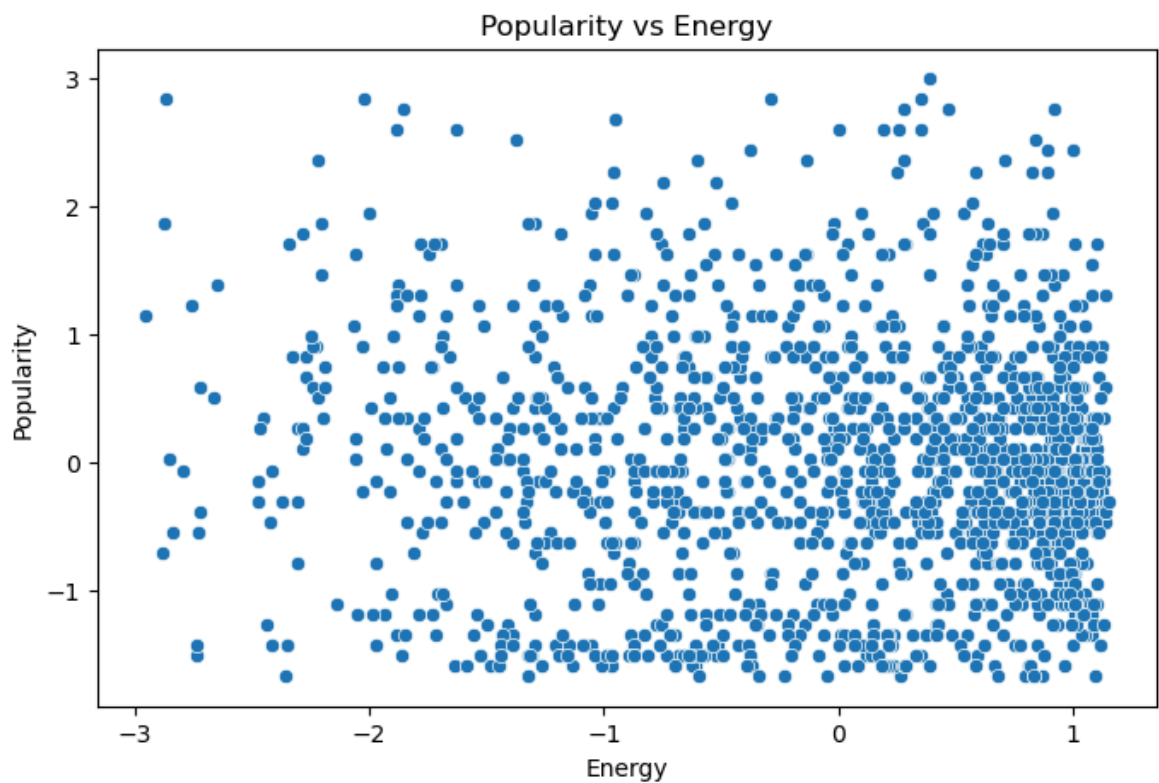
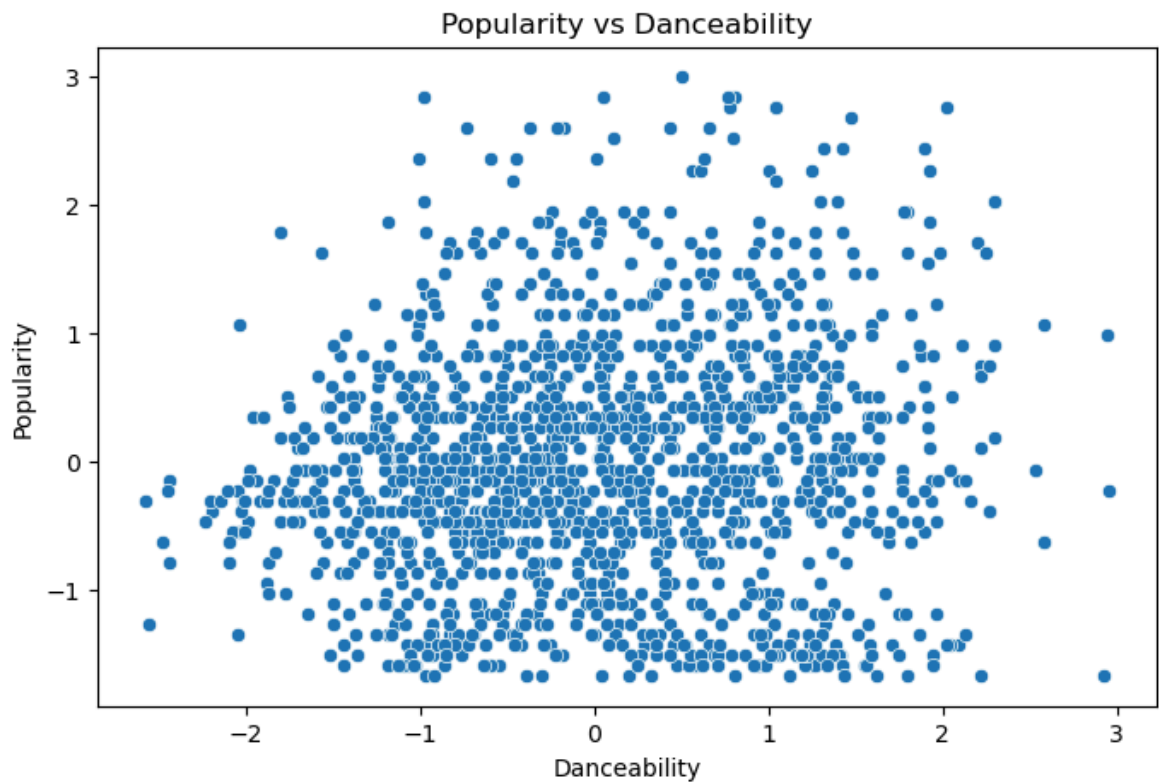
```
In [30]: plt.figure(figsize=(12,8))
sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm', square=True)
plt.title('Correlation Matrix of Song Features')
```

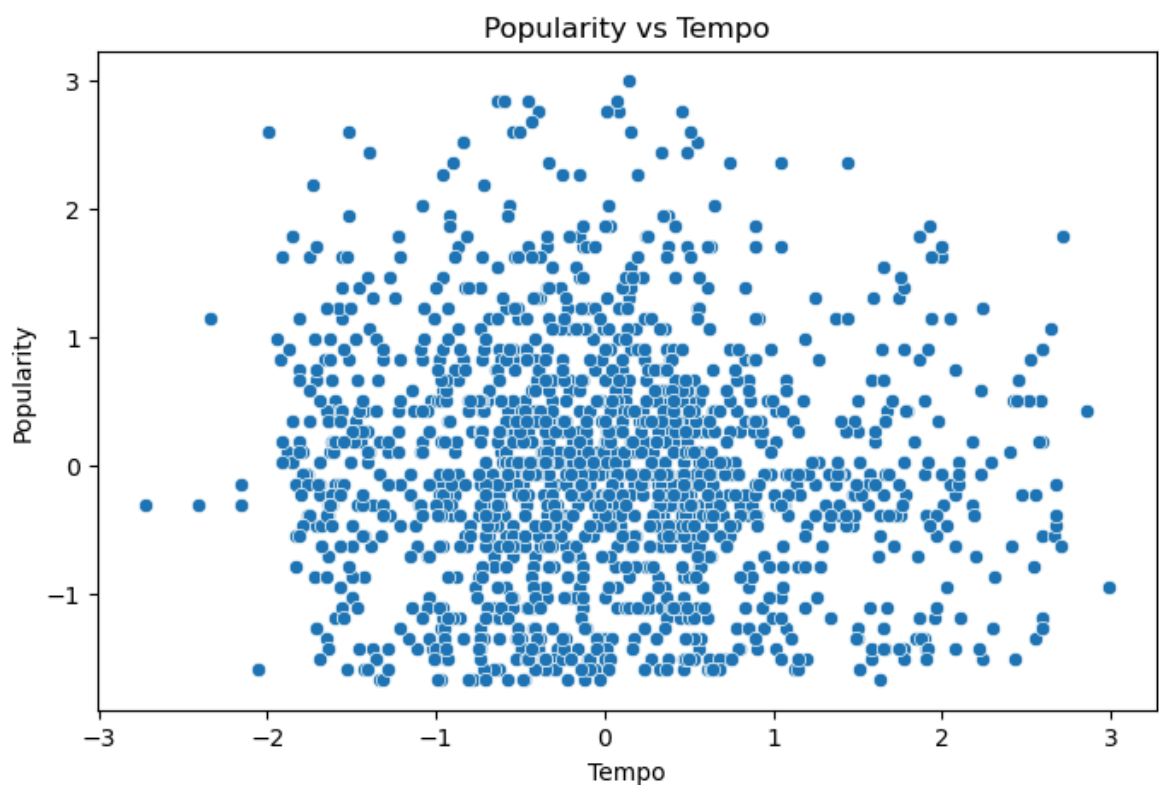
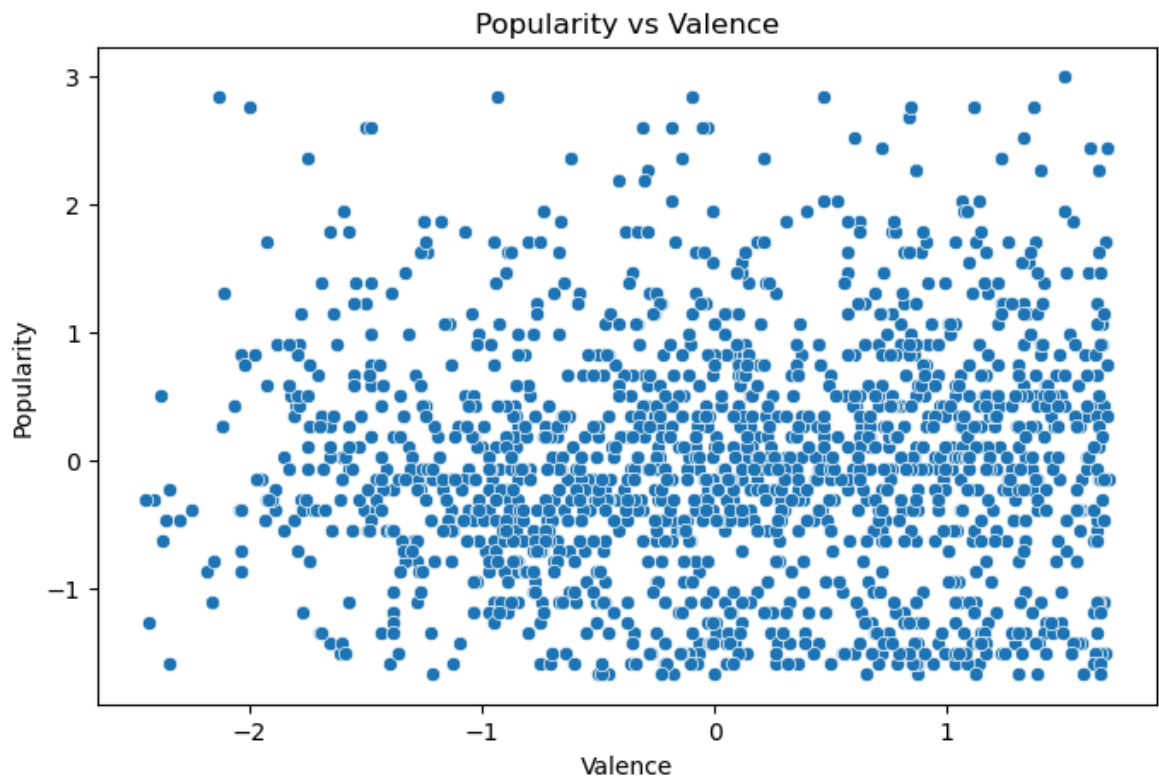
Out[30]: Text(0.5, 1.0, 'Correlation Matrix of Song Features')



c. Examine the relationship between a song's popularity and various factors, exploring how this correlation has evolved

```
In [34]: features = ['danceability', 'energy', 'valence', 'tempo']
for feature in features:
    plt.figure(figsize=(8,5))
    sns.scatterplot(data=songs_cohort_df, x=feature, y='popularity')
    plt.title(f'Popularity vs {feature.capitalize()}')
    plt.xlabel(feature.capitalize())
    plt.ylabel('Popularity')
```





d. Provide insights on the significance of dimensionality reduction techniques. Share your ideas and elucidate your observations

```
In [37]: numeric_features = songs_cohort_df.select_dtypes(include=['float64', 'int64'])
scaler = StandardScaler()
scaled_features = scaler.fit_transform(numeric_features)

pca = PCA(n_components=2)
principal_components = pca.fit_transform(scaled_features)
```

```

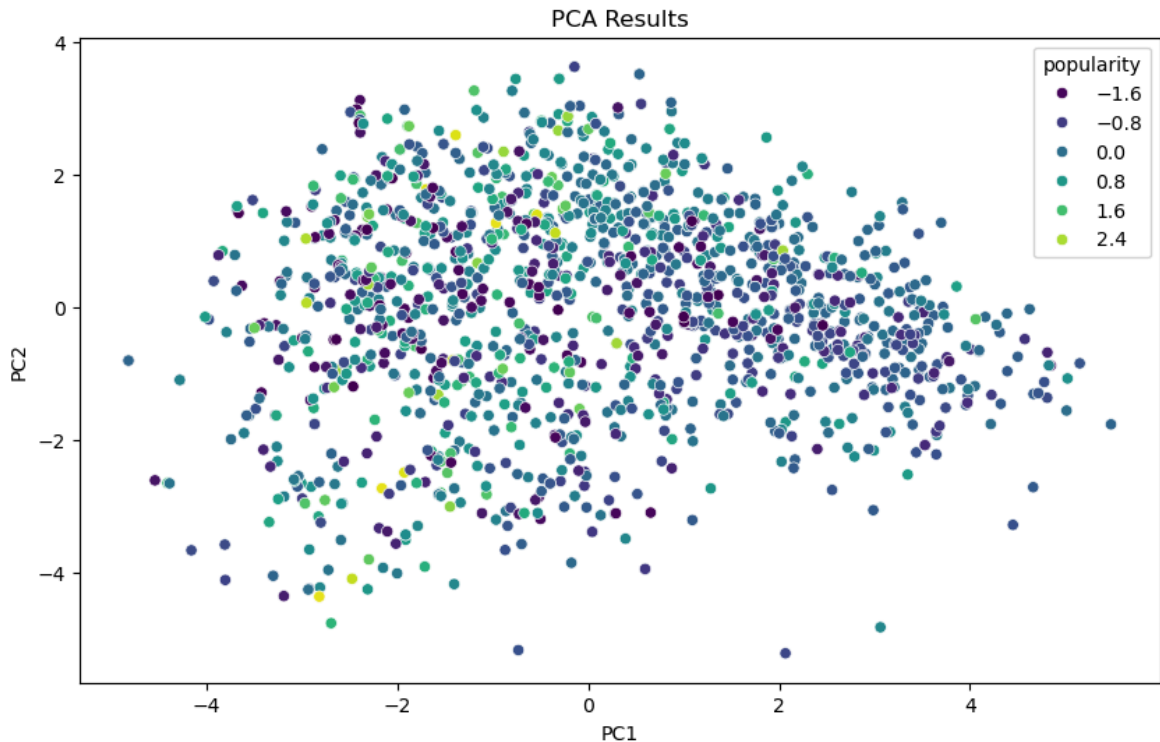
explained_variance = pca.explained_variance_ratio_
print(f'Explained Variance: {explained_variance}')

songs_cohort_df['PC1'] = principal_components[:, 0]
songs_cohort_df['PC2'] = principal_components[:, 1]

plt.figure(figsize=(10,6))
sns.scatterplot(x='PC1', y='PC2', data=songs_cohort_df, hue='popularity',
plt.title('PCA Results')
plt.show()

```

Explained Variance: [0.33898434 0.21160023]



Observation

- **First Principal Component (PC1):**

The first principal component (PC1) explains **30.68%** of the total variance in the dataset. This indicates that PC1 captures the largest source of variability, but no single component dominates the dataset.

- **Second Principal Component (PC2):**

The second principal component (PC2) explains an additional **15.39%** of the variance. Combined with PC1, the first two principal components explain **46.07%** of the total variance.

- **Cumulative Explained Variance:**

The cumulative explained variance of **46.07%** suggests that the first two components retain a significant amount of information while reducing the dimensionality of the dataset. However, more than half of the dataset's variance remains unexplained, indicating that additional principal components are required to preserve most of the original information.

- **Interpretation:**

Since the first two principal components explain less than **50%** of the total variance, the dataset is likely influenced by multiple underlying factors rather than a single dominant feature. This reflects the complexity of the data, where several variables contribute meaningfully to its overall structure.

- **Conclusion:**

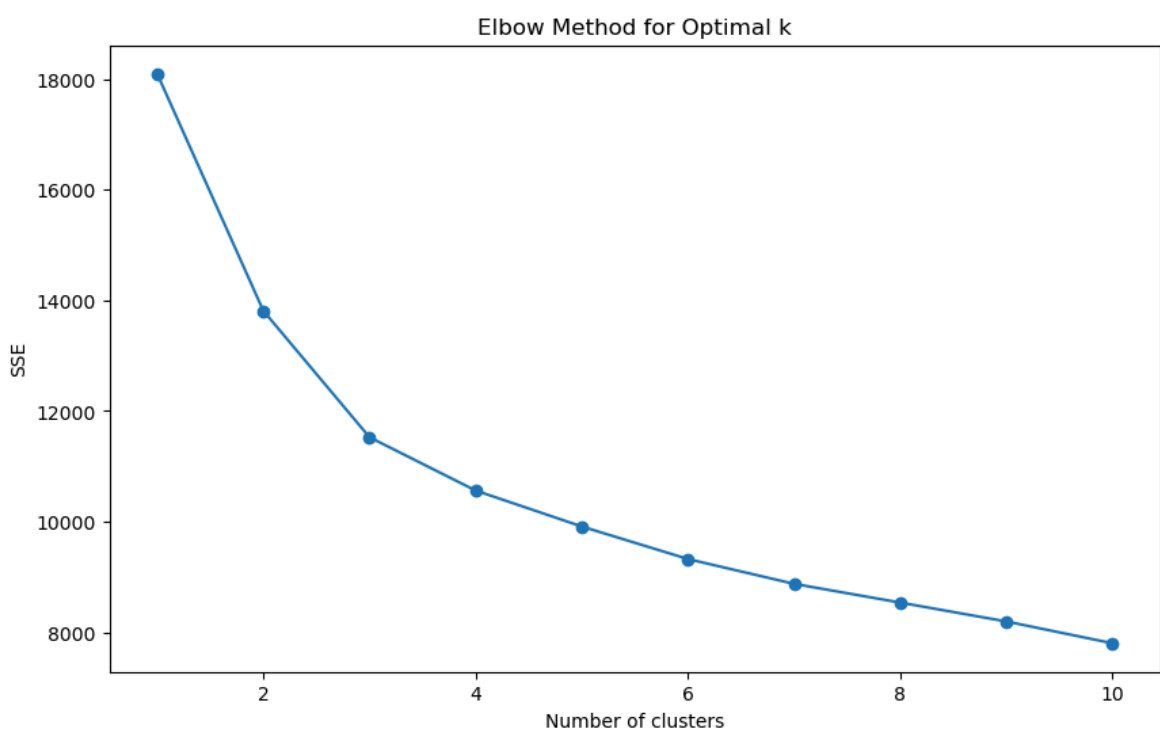
PCA successfully reduces the dimensionality of the dataset while retaining nearly half of its variability in just two components. For visualization, two principal components are sufficient to identify broad patterns and clusters. However, for downstream tasks such as predictive modeling or feature extraction, additional principal components may be needed to capture a higher percentage (e.g., **80–95%**) of the total variance.

4. Perform cluster analysis

a. Identify the right number of clusters

```
In [48]: # Elbow method
sse = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(scaled_features)
    sse.append(kmeans.inertia_)

plt.figure(figsize=(10,6))
plt.plot(range(1, 11), sse, marker='o')
plt.title('Elbow Method for Optimal k')
plt.xlabel('Number of clusters')
plt.ylabel('SSE')
plt.show()
```



b. Use appropriate clustering algorithms

```
In [49]: numeric_cols = [  
    'acousticness', 'danceability', 'energy', 'instrumentalness',  
    'liveness', 'loudness', 'speechiness', 'tempo', 'valence',  
    'popularity', 'duration_ms'  
]  
df_numeric = songs_cohort_df[numeric_cols]  
  
scaler = StandardScaler()  
df_scaled = scaler.fit_transform(df_numeric)  
df_scaled
```

```
Out[49]: array([[ 0.85755561, -1.03548347,  0.97594713, ..., -1.19833618,  
    1.17776377,  0.05469019],  
    [ 0.76355292, -0.61140044,  0.99892171, ..., -1.22017326,  
    1.17776377,  0.17354539],  
    [ 1.43947699, -0.7315573 ,  1.09082001, ..., -1.94516418,  
    1.00363094,  0.68195577],  
    ...,  
    [ 0.5621186 ,  2.244092 , -0.12108881, ...,  1.05961746,  
    0.82949812, -0.86279299],  
    [-0.12723443,  1.60796745, -1.43063956, ..., -0.26370934,  
    0.56829888, -1.51020695],  
    [ 0.61583442,  1.79880481,  0.79789418, ...,  1.6448511 ,  
    1.26483018, -0.70681859]], shape=(1508, 11))
```

```
In [50]: kmeans = KMeans(n_clusters=3, random_state=42)  
songs_cohort_df['cluster'] = kmeans.fit_predict(df_scaled)  
songs_cohort_df
```

Out [50]:

	name	album	release_date	track_number		id	
1	Street Fighting Man - Live	Licked Live In Nyc	2022-06-10	2	6GVgVJBKkGJoRfarYRvGTU	s	
2	Start Me Up - Live	Licked Live In Nyc	2022-06-10	3	1Lu761pZ0dBTGpzxaQoZNW	sp	
3	If You Can't Rock Me - Live	Licked Live In Nyc	2022-06-10	4	1agTQzOTUnGNggycEqiDH	s	
4	Don't Stop - Live	Licked Live In Nyc	2022-06-10	5	7piGJR8YndQBQWVXv6KtQw	sp	
5	Monkey Man - Live	Licked Live In Nyc	2022-06-10	6	3ixZ1NQk5sqvBZgWeldiqr		
...	...	...	...	...	...	...	
1605	Carol	The Rolling Stones	1964-04-16	8	08l7M5UpRnffGI0FyuRiQZ		
1606	Tell Me	The Rolling Stones	1964-04-16	9	3JZIIQBstM6WwoJdzFDLhx	s	
1607	Can I Get A Witness	The Rolling Stones	1964-04-16	10	0t2qvfSBQ3Y08lzRRoVTdb		
1608	You Can Make It If You Try	The Rolling Stones	1964-04-16	11	5ivls5vwSj0RChOlviY3On		
1609	Walking The Dog	The Rolling Stones	1964-04-16	12	43SkTJJ2xleDaeiE4TIM70		

1508 rows × 21 columns

c. Define each cluster based on the features

```
In [51]: cluster_summary = songs_cohort_df.groupby('cluster')[numeric_cols].mean()  
cluster_summary
```

Out [51]:

	acousticness	danceability	energy	instrumentalness	liveness	loudness
cluster						
0	-0.291367	0.572144	0.252495	0.304961	-0.504799	0.2229
1	0.658509	0.293250	-1.175302	-0.296077	-0.606540	-0.8184
2	-0.297948	-0.743437	0.733825	-0.069041	0.954956	0.5432

In [52]:

```
most_common_album = songs_cohort_df.groupby('cluster')['album'].agg(lambda x: x.mode()[0])
cluster_summary['common_album'] = most_common_album
cluster_summary
```

Out [52]:

	acousticness	danceability	energy	instrumentalness	liveness	loudness
cluster						
0	-0.291367	0.572144	0.252495	0.304961	-0.504799	0.2229
1	0.658509	0.293250	-1.175302	-0.296077	-0.606540	-0.8184
2	-0.297948	-0.743437	0.733825	-0.069041	0.954956	0.5432

Cluster Analysis

Cluster 0

Representative Album: *Honk (Deluxe)*

Characteristics:

- Low acousticness
- High danceability
- Moderately high energy
- High instrumentalness
- Low liveness
- Slightly higher loudness
- Low speechiness
- Slightly slower tempo
- Very high valence (more positive/happy songs)
- Highest popularity
- Slightly shorter song duration

Interpretation: This cluster represents energetic, upbeat, and highly danceable songs with positive moods. These tracks are commercially successful and appear to be polished studio recordings with broad audience appeal.

Cluster 1

Representative Album: *Aftermath (UK Version)*

Characteristics:

- Highest acousticness
- Moderate danceability
- Lowest energy
- Low instrumentalness
- Low liveness
- Lower loudness
- Low speechiness
- Slower tempo
- Slightly negative valence
- Moderate popularity
- Slightly shorter duration

Interpretation: This cluster contains softer, acoustic-oriented songs with lower energy and quieter production. These tracks are suitable for relaxed listening and exhibit moderate commercial popularity.

Cluster 2

Representative Album: *Voodoo Lounge Uncut (Live)*

Characteristics:

- Low acousticness
- Lowest danceability
- Highest energy
- Moderate instrumentalness
- Highest liveness
- Highest loudness
- Highest speechiness
- Fastest tempo
- Lowest valence (more intense or darker mood)
- Lowest popularity
- Longest song duration

Interpretation: This cluster consists of energetic, live-performance-oriented tracks with high tempo and loudness. Despite their intensity, they are less danceable and have lower popularity, suggesting they cater to a more niche audience.

Overall Insights

- **Cluster 0** represents **commercially successful, upbeat, danceable songs** with the highest positive mood and popularity.
- **Cluster 1** groups **acoustic, mellow, and low-energy tracks**, making them suitable for calm and relaxed listening.

- **Cluster 2** consists of **high-energy live performances** characterized by loudness, fast tempo, and strong stage presence, but with lower mainstream popularity.

The clustering effectively separates songs into three distinct musical styles based on their audio characteristics, demonstrating that features such as **acousticness**, **danceability**, **energy**, **tempo**, **valence**, and **liveness** are key factors in distinguishing different types of songs.

In []: